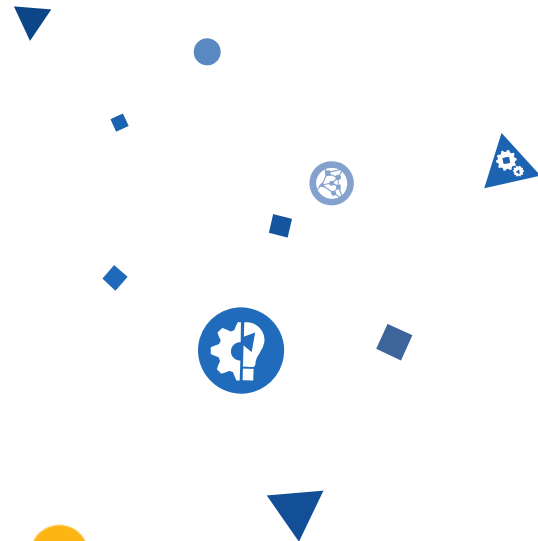




Architecting for performance. A top-down approach.



Prague, 19-20 October 2017

www.luxoft.com

Ionuț Baloșin

Software Architect

 @ionutbalosin

About Me

Ionuț Baloșin

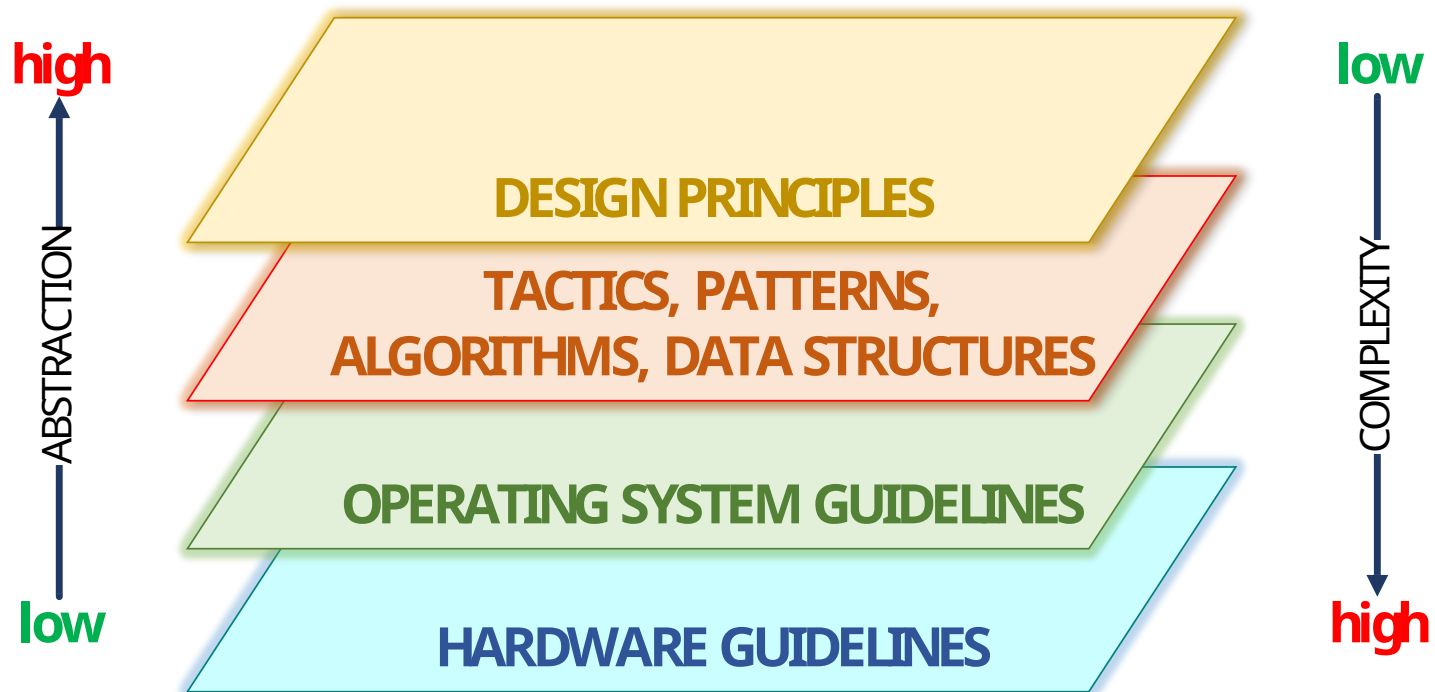
Software Architect @ luxoft.com

- **10+** years of software development experience

Technical Trainer

- Java Performance and Tuning
- Software Architecture

Agenda



My Latency Hierarchical Model

Ultra-low Latency

(< 1ms)

Low Latency

(~ ten of ms)

Affordable Latency

(~ hundreds of ms)

Performance is not an ASR*

(~ sec)

What is **Performance**?

“Performance it’s about time and the software system’s ability to meet timing requirements.”

“Software Architecture in Practice” - Rick Kazman, Paul Clements, Len Bass



Does IT Industry Need Better Namings?



| Posted by [Ionut Balosin](#) on Apr 24, 2017. Estimated reading time: 13 minutes

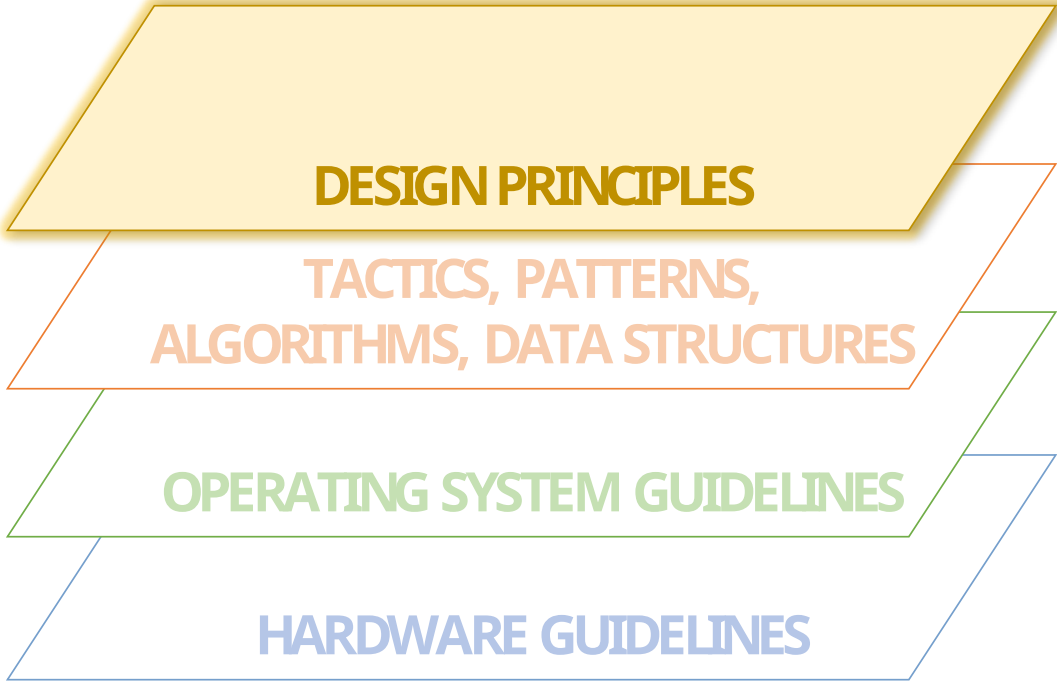
Non-functional requirements or quality attributes

Oftentimes, business requirements are drilled down in functional and non-functional specifications. In this context, the non-functional requirements refer to quality attributes such as performance, availability, scalability, security, usability, etc.

But let's try to better understand what's behind the non-functional terminology guided by two approaches: the first one is to search the word in the dictionary and the second one is to show the technical implications with regards to their achievability.

According to the dictionary, the "*non*" word means "*not, absence of, unimportant, worthless*". Based on this definition, the question which arises is: how can we name something non-functional and still having an impact on the architecture, since by semantics it is "worthless"? Why shouldn't we drop or put aside these requirements? Hence, the first inconsistency.

Following with the second approach, all non-functional requirements (e.g. security, usability) are implemented through functions (e.g. security via encryption, usability via a wizard), so they rely on concrete functions. It leads to the second question: how can we name something as being non-



DESIGN PRINCIPLES

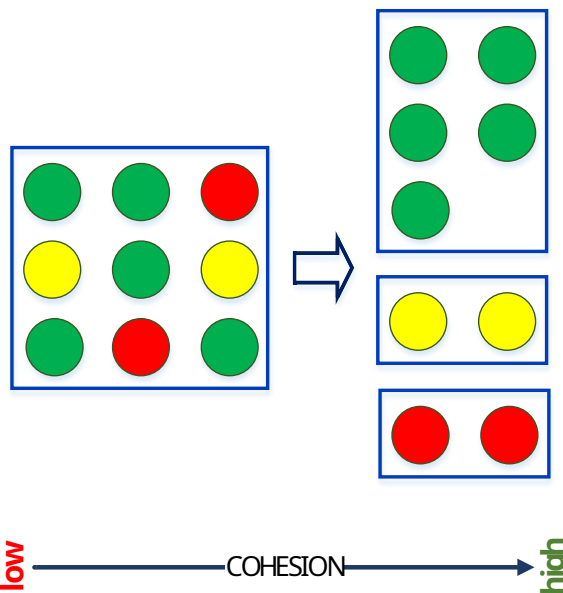
**TACTICS, PATTERNS,
ALGORITHMS, DATA STRUCTURES**

OPERATING SYSTEM GUIDELINES

HARDWARE GUIDELINES

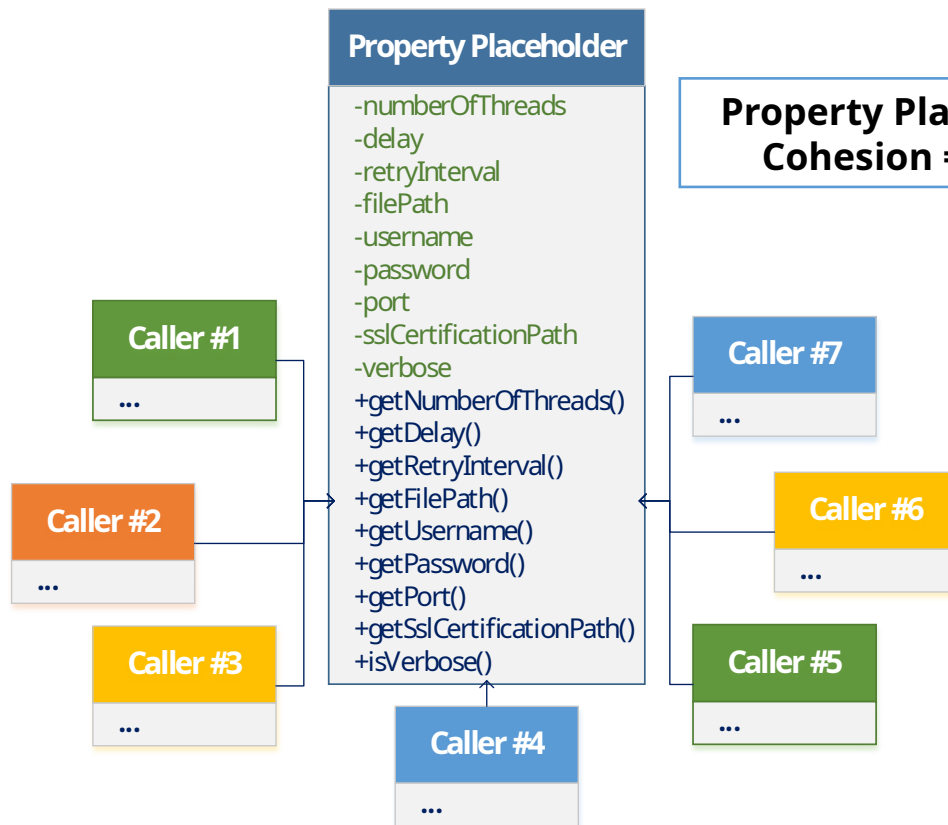
Cohesion

Cohesion represents the degree to which the elements inside a module work / belong together.



Cohesion => Better locality => CPU iCache / dCache friendly

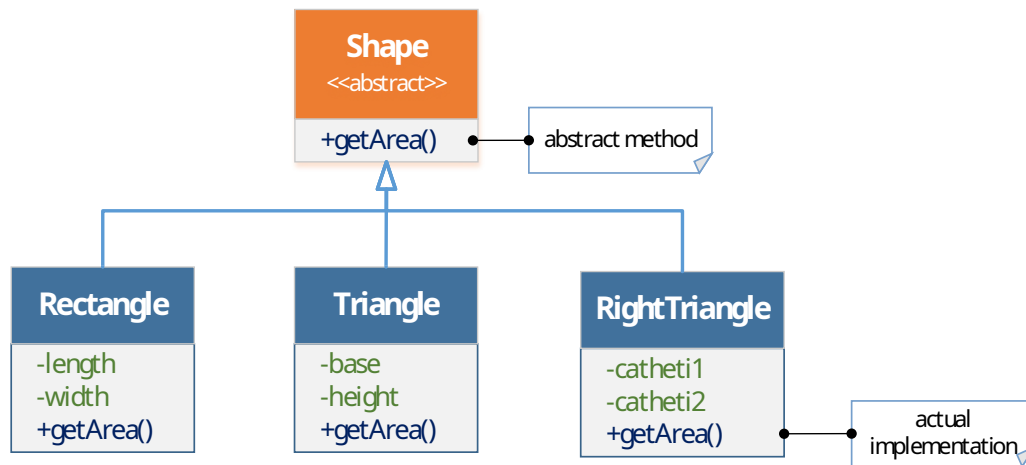
Property Placeholder



Property Placeholder => Low Object Cohesion => CPU dCache misses

Abstractions

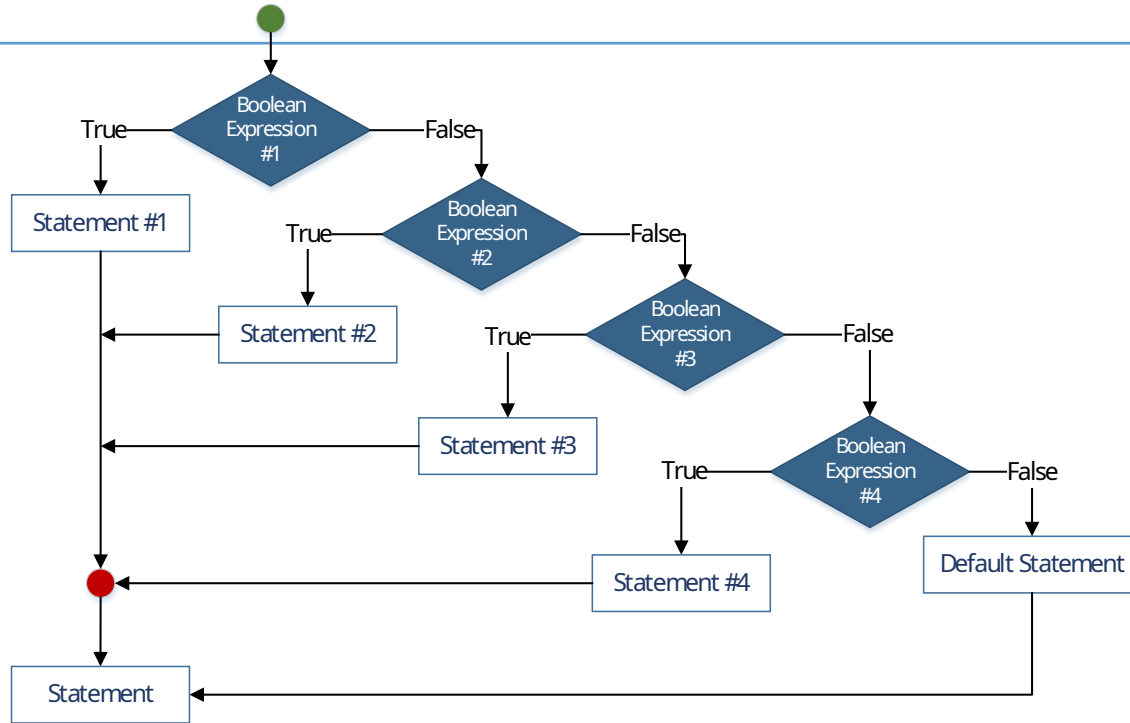
"The purpose of abstracting is not to be vague, but to create a new semantic level in which one can be absolutely precise" - **Edsger Dijkstra**



Abstractions => Polymorphism (e.g. Virtual Calls) => Increased runtime cost

Cyclomatic Complexity

Cyclomatic complexity is the number of linearly independent paths through a program's source code.

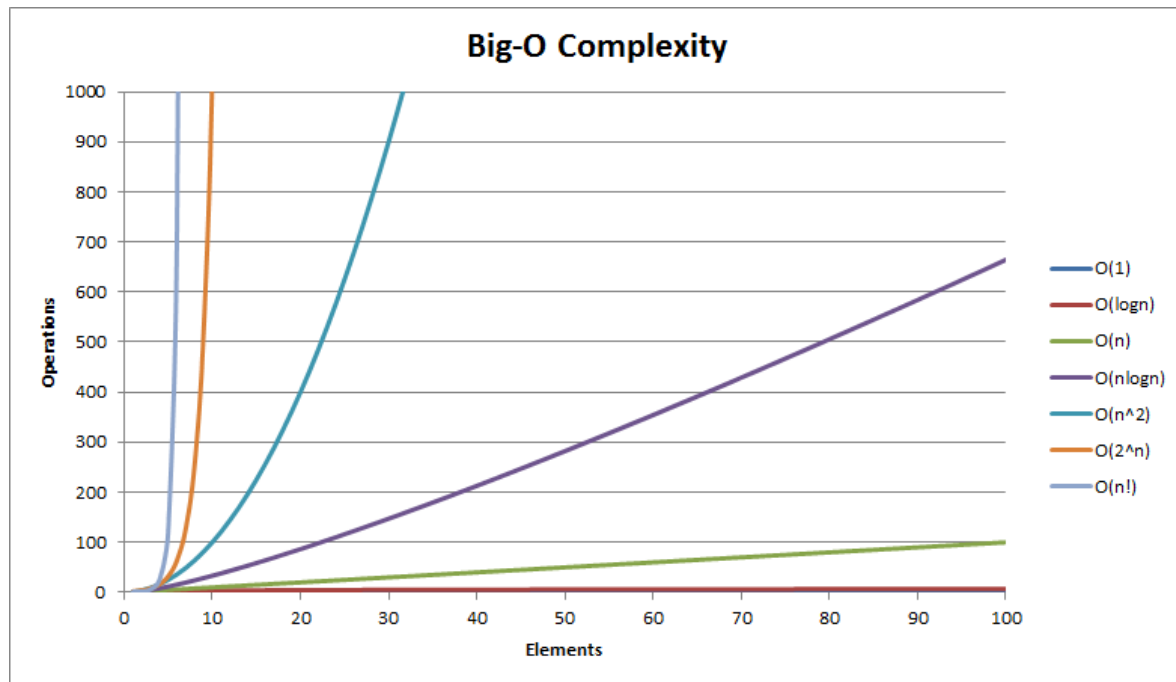


Higher Cyclomatic Complexity => Branch miss predictions => Pipeline stalls

Cyclomatic Complexity

Recommendation

- help the processor to make good prefetching decisions (e.g. code layout with more “predictable” branches)



[Source: <https://stackoverflow.com/questions/29927439/>]

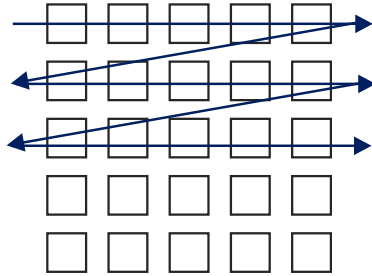
Service Time is a measure of algorithm complexity

Algorithms Complexity

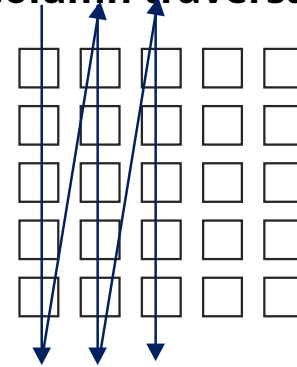
But ... is it all about **Big-O Complexity**?

Matrix Traversal

Row traversal



Column traversal



Matrix Traversal

Row traversal



```
public long rowTraversal() {  
    long sum = 0;  
    for (int i = 0; i < mSize; i++)  
        for (int j = 0; j < mSize; j++) {  
            sum += matrix[i][j];  
        }  
  
    return sum;  
}
```

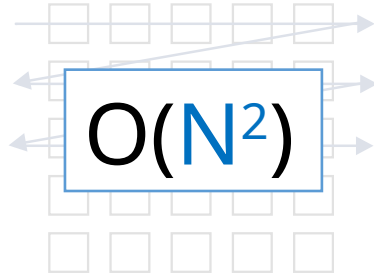
Column traversal



```
public long columnTraversal() {  
    long sum = 0;  
    for (int i = 0; i < mSize; i++)  
        for (int j = 0; j < mSize; j++) {  
            sum += matrix[j][i];  
        }  
  
    return sum;  
}
```

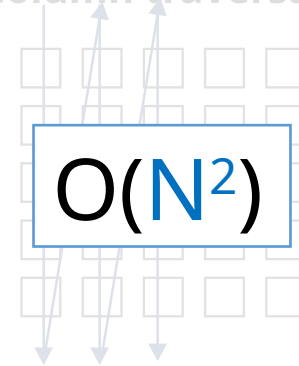
Matrix Traversal

Row traversal



```
public long rowTraversal() {  
    long sum = 0;  
    for (int i = 0; i < mSize; i++)  
        for (int j = 0; j < mSize; j++) {  
            sum += matrix[i][j];  
        }  
  
    return sum;  
}
```

Column traversal



```
public long columnTraversal() {  
    long sum = 0;  
    for (int i = 0; i < mSize; i++)  
        for (int j = 0; j < mSize; j++) {  
            sum += matrix[j][i];  
        }  
  
    return sum;  
}
```

Matrix Traversal

Matrix size	Row Traversal (ij) (ops/ μ s)	Column Traversal (ji) (ops/ μ s)
64 x 64	0.773	0.409
512 x 512	0.012	0.003
1024 x 1024	0.003	0.001
4096 x 4096	10^{-4}	10^{-5}
	$O(N^2)$	$O(N^2)$

higher is better

Matrix Traversal

Matrix size	Row Traversal (ij) (ops/ μ s)	Column Traversal (ji) (ops/ μ s)
64 x 64	0.773	0.409
512 x 512	0.012	0.003
1024 x 1024	0.003	0.001
4096 x 4096	10^{-4}	10^{-5}
	$O(N^2)$	$O(N^2)$

higher is better

Matrix Traversal

Why such noticeable difference ?

~ 1 order of magnitude

Matrix Traversal

Matrix size 4096 x 4096	Row Traversal ij	Column Traversal ji
cycles per instruction	0.849	1.141
L1-dcache-loads	$10^9 \times 0.056$	$10^9 \times 9.4$
L1-dcache-load-misses	$10^9 \times 0.019$	$10^9 \times 6.0$
LLC-loads	$10^9 \times 0.014$	$10^9 \times 6.1$
LLC-load-misses	$10^9 \times 0.004$	$10^9 \times 0.084$
dTLB-loads	$10^9 \times 0.026$	$10^9 \times 9.4$
dTLB-load-misses	$10^3 \times 13$	$10^3 \times 101$

lower is better

Matrix Traversal

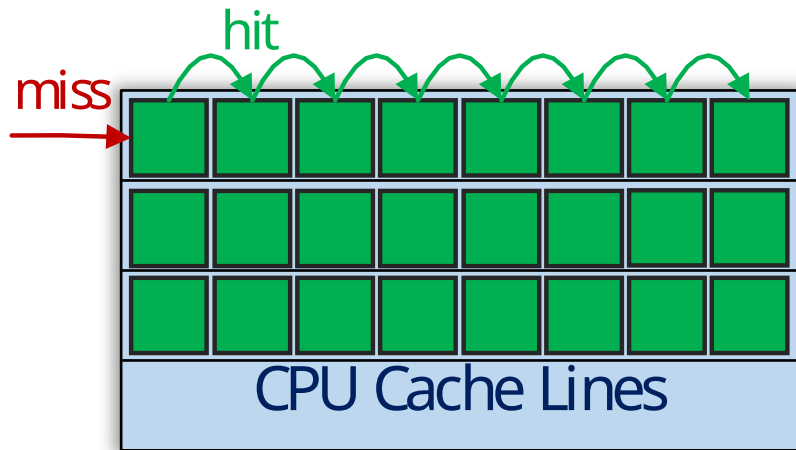
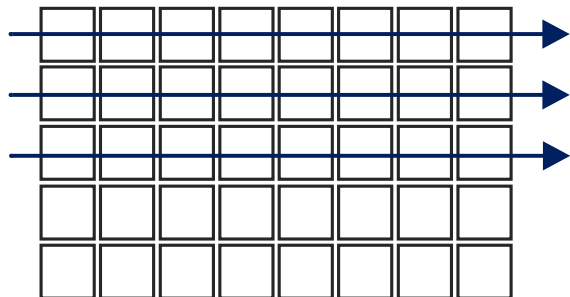
Matrix size 4096 x 4096	Row Traversal ij	Column Traversal ji
cycles per instruction	0.849	1.141
L1-dcache-loads	$10^9 \times 0.056$	$10^9 \times 9.4$
L1-dcache-load-misses	$10^9 \times 0.019$	$10^9 \times 6.0$
LLC-loads	$10^9 \times 0.014$	$10^9 \times 6.1$
LLC-load-misses	$10^9 \times 0.004$	$10^9 \times 0.084$
dTLB-loads	$10^9 \times 0.026$	$10^9 \times 9.4$
dTLB-load-misses	$10^3 \times 13$	$10^3 \times 101$

lower is better

~ 1 or 2 orders of magnitude difference for dCache misses

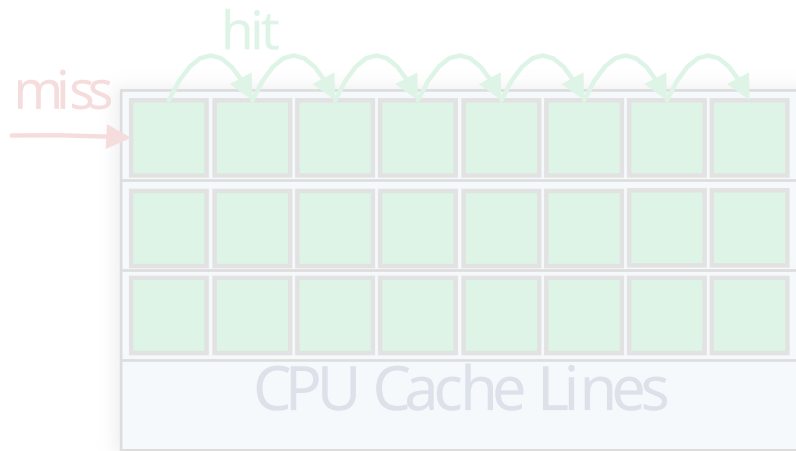
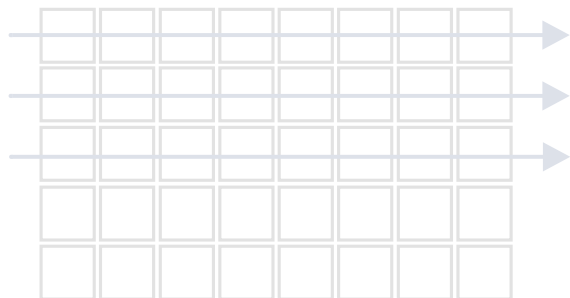
Matrix Traversal

Row traversal

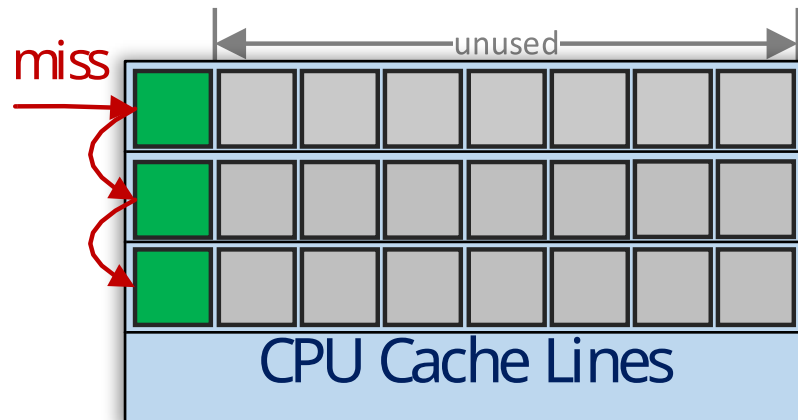
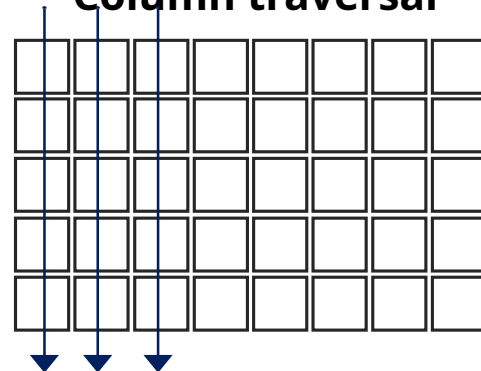


Matrix Traversal

Row traversal



Column traversal



Algorithms Complexity

On modern architectures **Service Time is highly impacted by CPU caches**

Algorithms Complexity

Big-O Complexity might win for huge data sets where CPU caches could not help

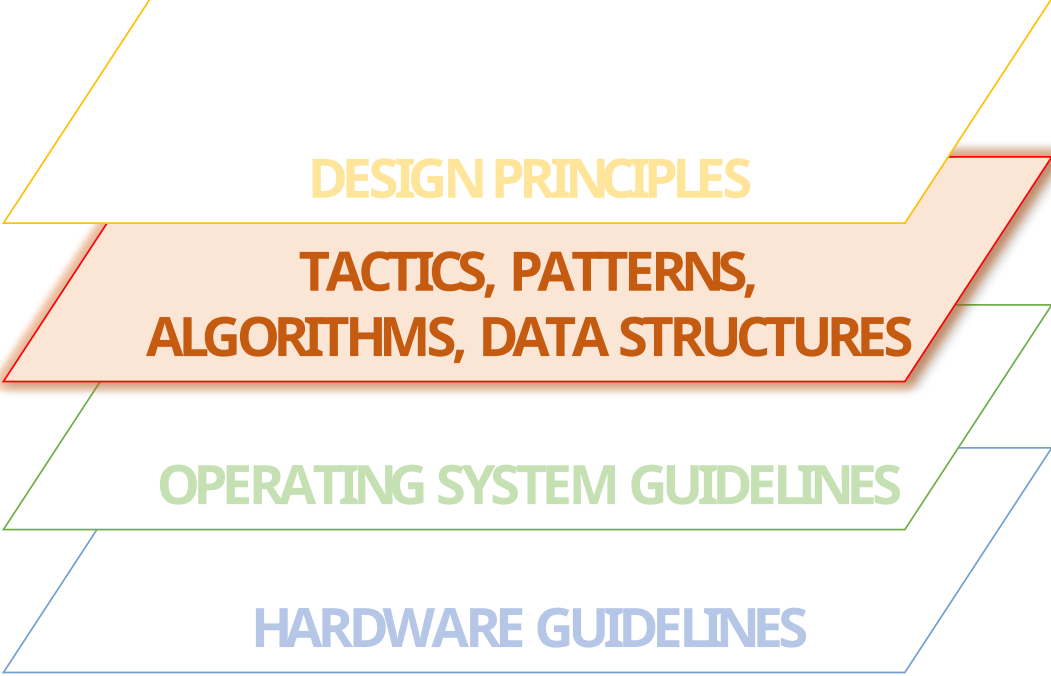
Complexity

Recommendation

- reduce the code footprint as possible (e.g. small methods)

Complexity of the problem might impact code quality hence increases the iCache

Compiler optimizations (e.g. loop unrolling, inlining) might affect code footprint



DESIGN PRINCIPLES

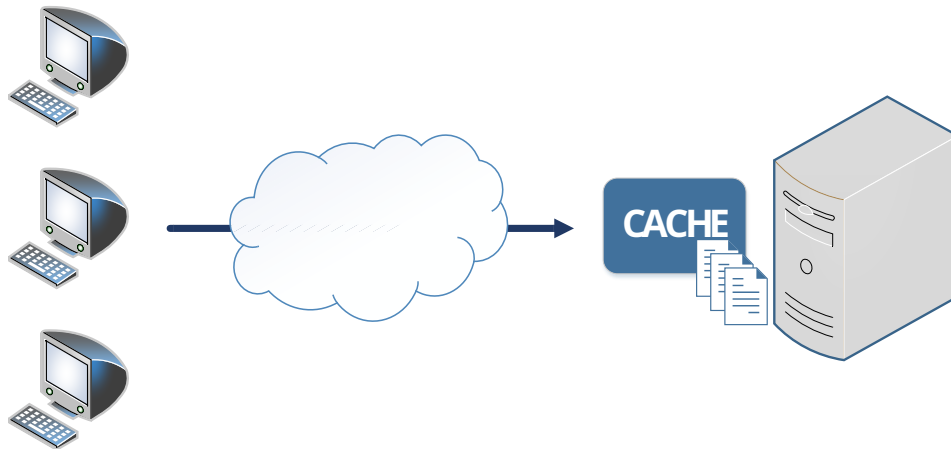
**TACTICS, PATTERNS,
ALGORITHMS, DATA STRUCTURES**

OPERATING SYSTEM GUIDELINES

HARDWARE GUIDELINES

Caching

Caching stores application data in an optimized location to facilitate faster and easier retrieval



Data Patterns (e.g. read/write through, write behind, read ahead)

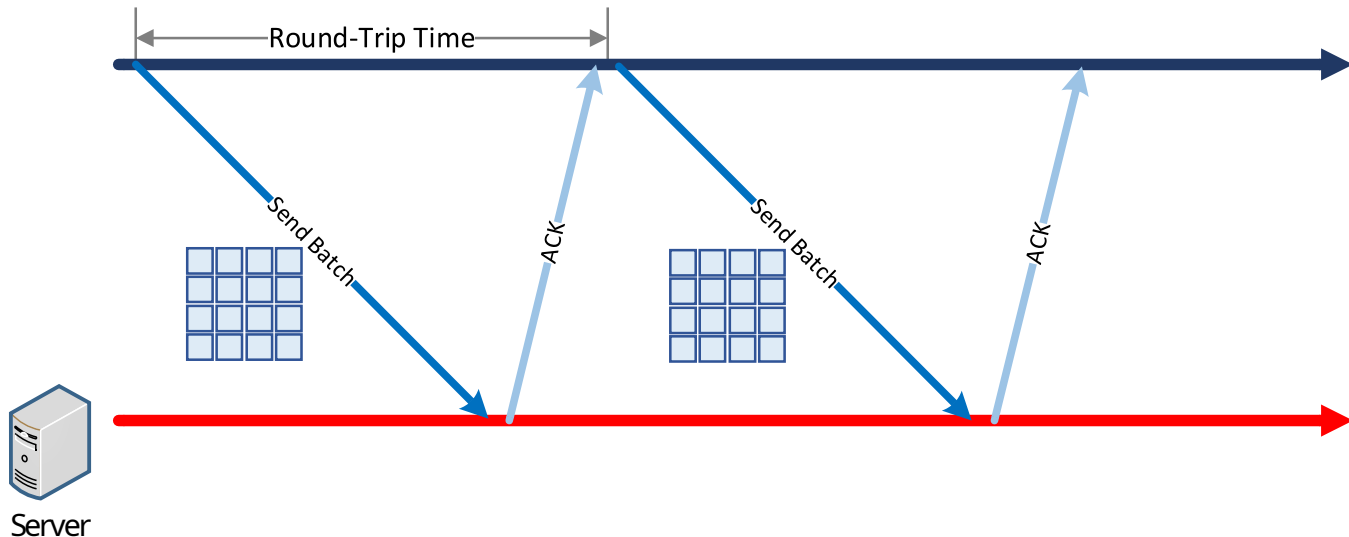
Eviction Algorithm (e.g. LRU, LFU, FIFO)

Fetching Strategy (e.g. pre-fetch, on-demand, predictive)

Topology (e.g. local, partitioned/distributed, partitioned-replicated)

Batching

Batching minimizes the number of server round trips, especially when data transfer is long.

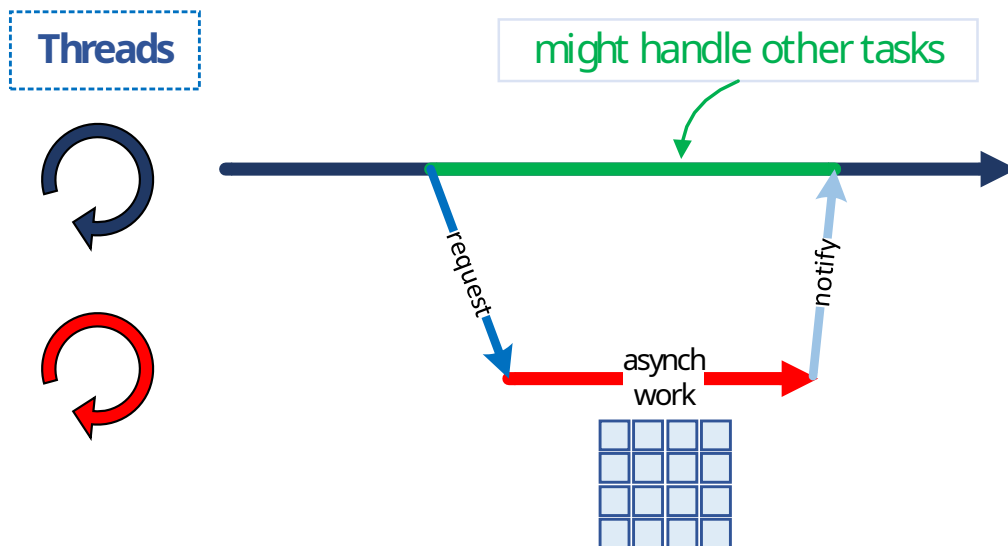


Solution is limited by **bandwidth** and **Receiver's handling rate**

Find the suitable batch size for an optimal transfer (e.g. RTT, bandwidth)

Design Asynchronous

"Design asynchronous by default, make it synchronous when it is needed" - **Martin Thompson**



Designing asynchronous and stateless is a good recipe for performance !

Design Asynchronous

In Java

```
java.util.concurrent.CompletableFuture<T>  
    <U> CompletableFuture<U> supplyAsync(Supplier<U> supplier)
```

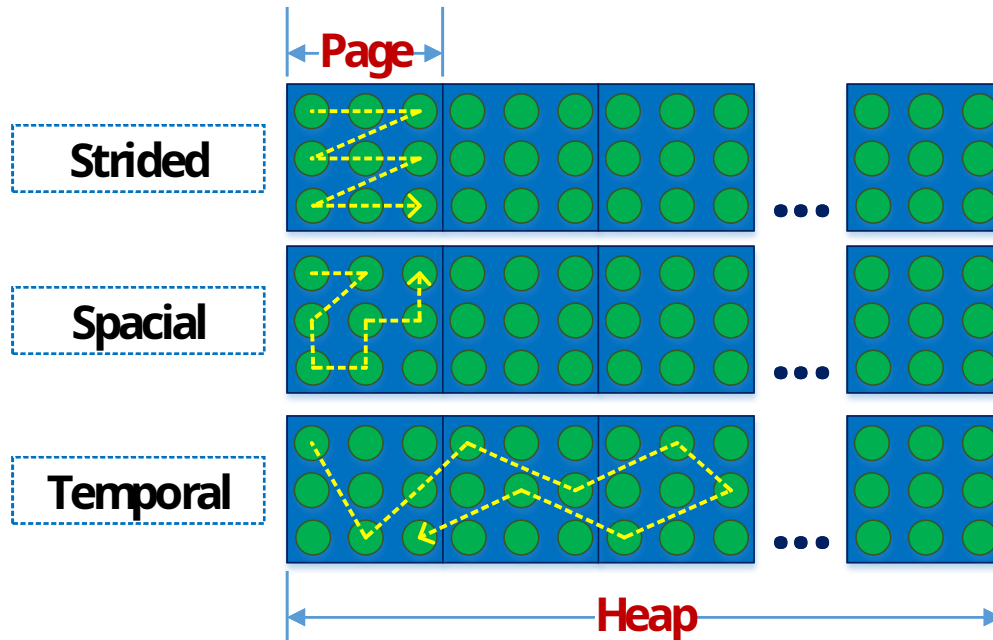
```
java.util.concurrent.Future<V>  
    boolean isDone()  
    V get()
```

```
java.util.concurrent.Flow.Publisher<T>  
    void subscribe(Flow.Subscriber<? super T> subscriber)
```

```
java.util.concurrent.Flow.Subscriber<T>  
    void onSubscribe(Flow.Subscription subscription)  
    void onComplete()  
    void onNext(T item)
```

```
java.util.concurrent.Flow.Subscription  
    void request(long n)
```

Memory Access Patterns

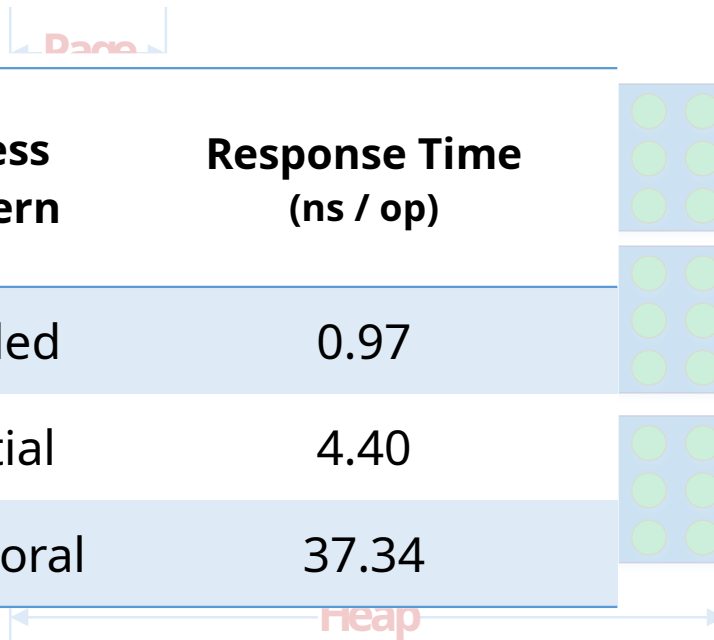


Strided - memory access is likely to follow a predictable pattern

Spatial - nearby memory is likely to be required soon

Temporal - memory accessed recently will likely be required again soon

Memory Access Patterns



The diagram illustrates memory access patterns on a 2D grid of 6x6 green circles. A blue horizontal bar highlights the 'Strided' pattern, which moves horizontally across the first row. A red double-headed arrow labeled 'Data' is positioned above the bar, and a red double-headed arrow labeled 'Heap' is positioned below it. The 'Spatial' pattern is indicated by a dashed box on the left, and the 'Temporal' pattern is indicated by a dashed box on the right.

Str	Access Pattern	Response Time (ns / op)
Sp	Strided	0.97
	Spatial	4.40
Tem	Temporal	37.34

Sequential - memory access is likely to follow a predictable pattern

Spatial - adjacent memory is likely to be required soon

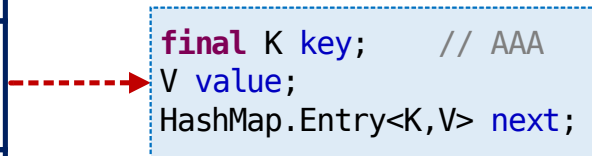
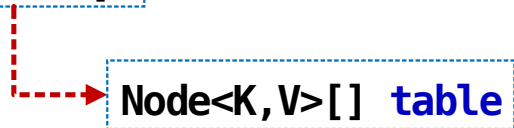
Temporal - memory accessed recently will likely be required again soon

HashMap

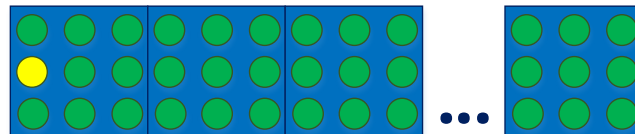
Chaining vs. Open Addressing

Data Structures

HashMap - chaining

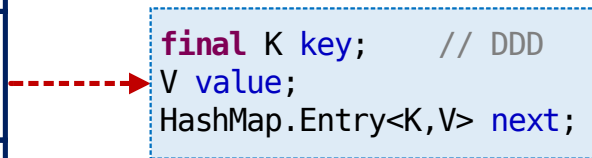
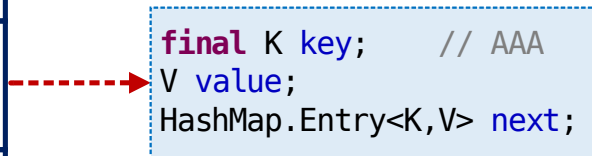
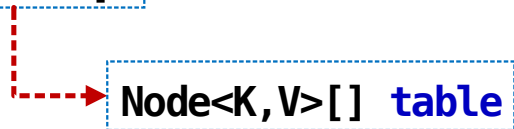


Memory Layout

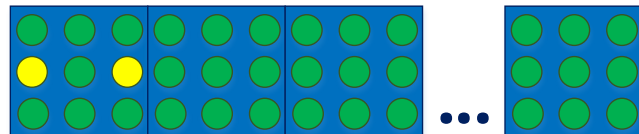


Data Structures

HashMap - chaining



Memory Layout



Data Structures

HashMap - chaining

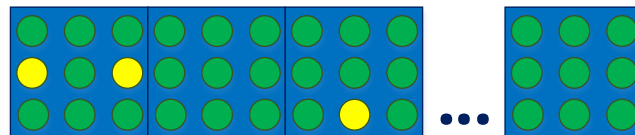
`Node<K, V>[] table`



`final K key; // AAA`
`V value;`
`HashMap.Entry<K, V> next;`

`final K key; // DDD`
`V value;`
`HashMap.Entry<K, V> next;`

Memory Layout



`final K key; // AAA`
`V value;`
`HashMap.Entry<K, V> next;`

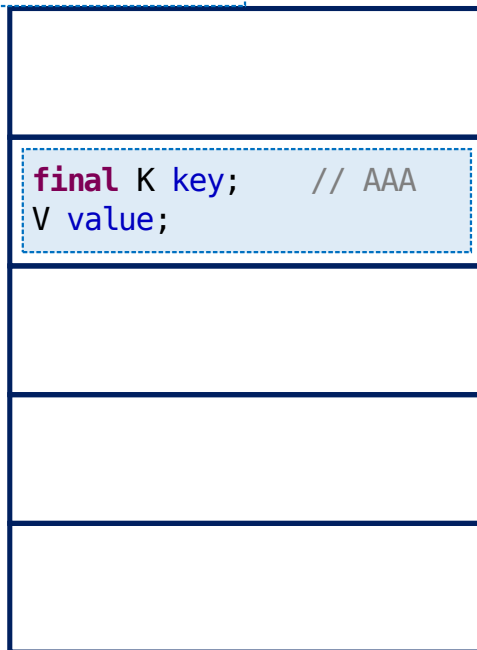
collision resolved by
separate chaining

Collisions – Temporal memory access

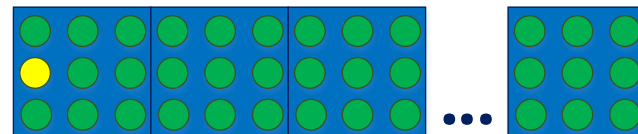
Data Structures

HashMap - open addressing

Node<K, V>[] **table**



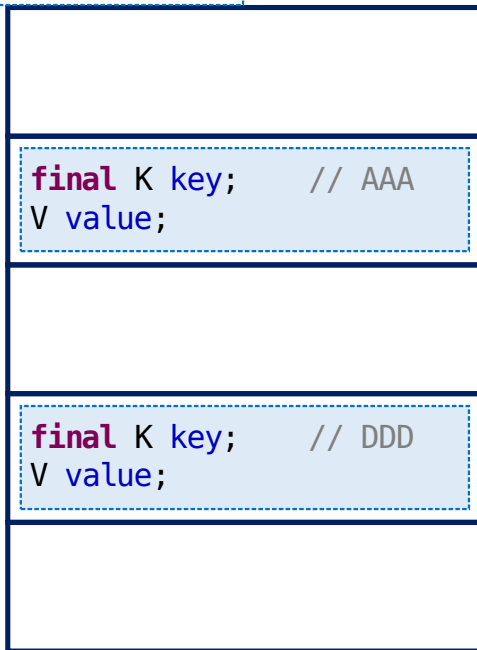
Memory Layout



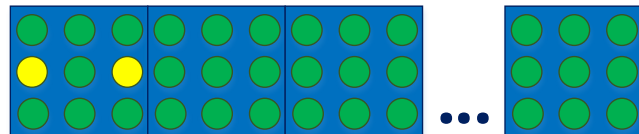
Data Structures

HashMap - open addressing

Node<K, V>[] **table**



Memory Layout

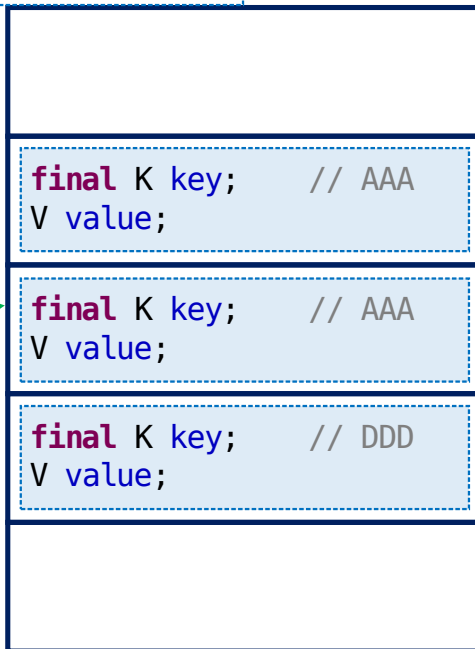


Data Structures

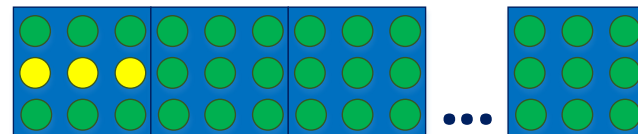
HashMap - open addressing

Node<K, V>[] table

collision resolved by
open addressing
with linear probing
($internal = 1$;
 $hash(x) + 1$)



Memory Layout



Collisions – **Sequential** memory access

Chaining vs. Open Addressing

	Chaining	Open Addressing
Collision resolution	Use external data structure (e.g. LinkedList)	Use the hash table itself
Memory waste	Pointer size overhead	No overhead, better locality
Performance dependence on Load Factor	Directly proportional	Proportional to $(LF) / (1 - LF)$
Implementation	Simple	Quite tricky

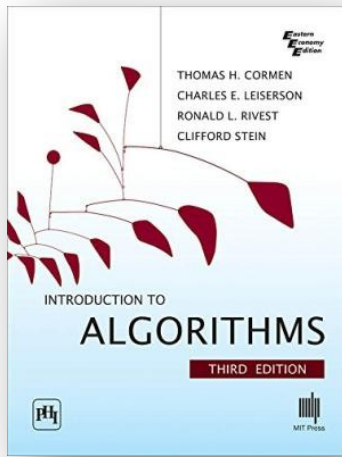
Open Addressing might be an alternative from performance standpoint

Data Structures

*“... the **choice of data structures** often **makes a big difference**. Most programmers just grab whatever List they used last time rather than thinking which implementation is the right one ...”*



Martin Fowler

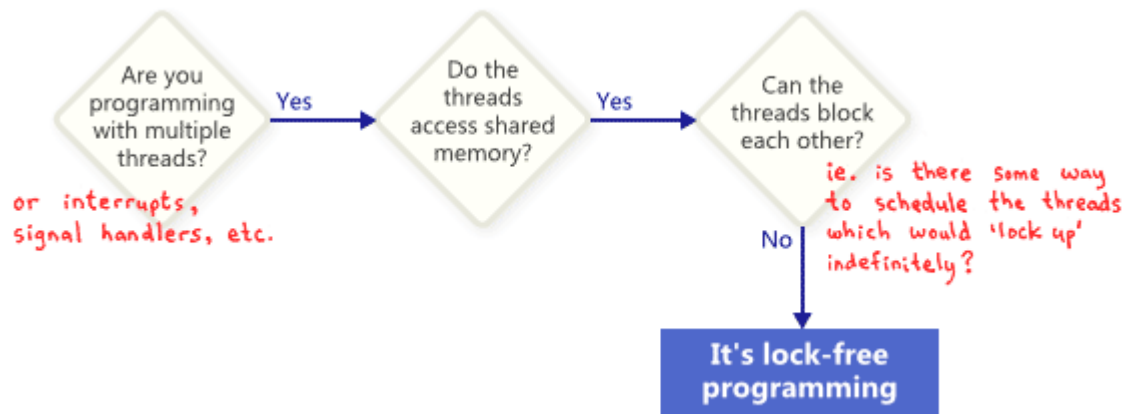


Articles by **Ulrich Drepper**

- ✓ What every programmer should know about memory
- ✓ CPU caches
- ✓ Virtual memory
- ✓ NUMA systems
- ✓ What programmers can do - cache optimization
- ✓ What programmers can do - multi-threaded optimizations
- ✓ Memory performance tools

Lock Free Algorithms

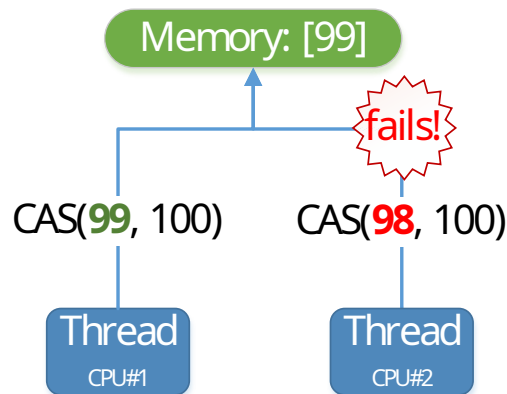
Lock Free - failure or suspension of any thread cannot cause failure or suspension of another thread and there is a guaranteed system-wide progress



[Source: <http://preshing.com/20120612/an-introduction-to-lock-free-programming>]

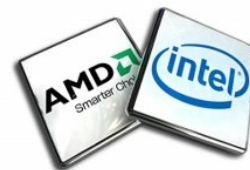
Lock Free Algorithms

Compare-And-Swap (CAS) - atomically update a memory location by another value if the previous value is the expected one



[lock] **CMPXCHG** reg, reg/mem

x86
x64



Lock Free Algorithms

Compare-And-Swap APIs

j.u.c.atomic.AtomicT

boolean compareAndSet(**T** expect, **T** update)

Data Structures using Compare-And-Swap

j.u.c.atomic.AtomicT

T getAndIncrement()

T getAndDecrement()

T getAndAdd(**T** delta)

T getAndSet(**T** newValue)

j.u.c.locks.ReentrantLock

void lock()

boolean tryAcquire(**int** acquires)

j.u.c.ConcurrentLinkedQueue

boolean casItem(**E** cmp, **E** val)

boolean casNext(Node<**E**> cmp, Node<**E**> val)

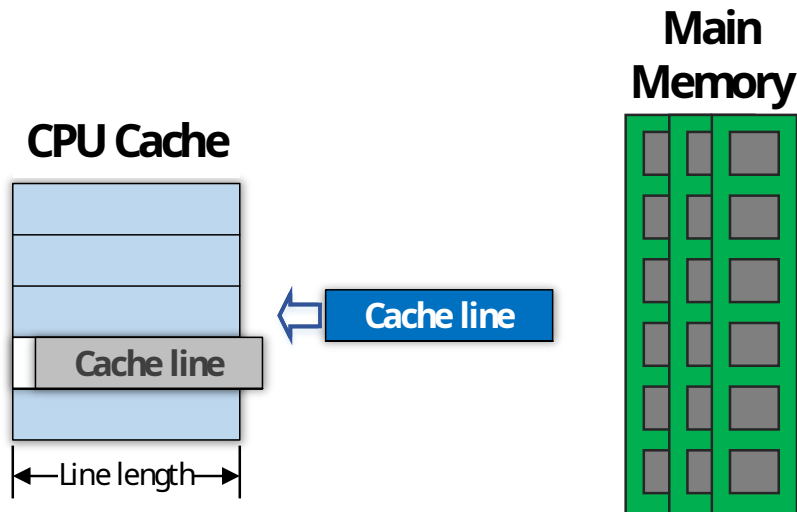
void updateHead(Node<**E**> h, Node<**E**> p)

boolean offer(**E** e)

boolean addAll(Collection<? **extends E**> c)

Cache Oblivious Algorithms

Cache Oblivious Algorithms take advantage of a CPU cache without having the size of the cache (or the length of the cache lines, etc.) as an explicit parameter



possible Cache miss effect

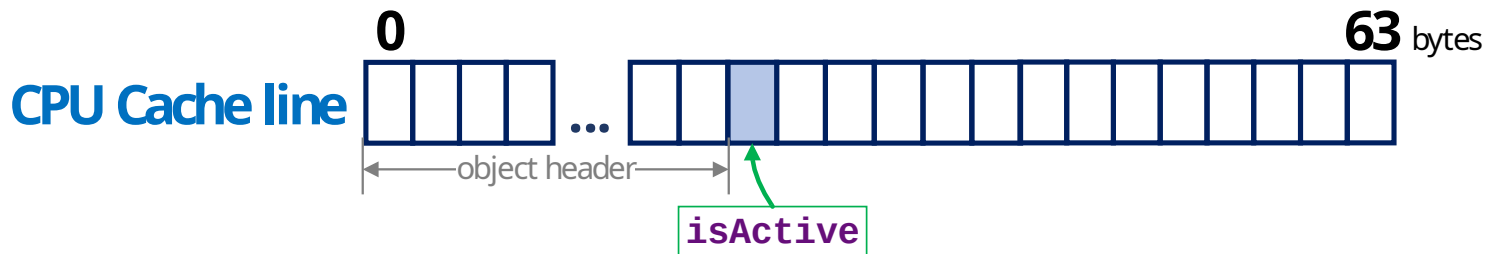
Examples: matrix multiplication, matrix transposition, sorting (funnel, distribution), etc.

Object Oriented Programming

```
public class Account {                                // assume sizeof(Account) 64 bytes
    private boolean isActive;
    private double amount;
    private String username;
    // ... other Fields ...
    // ... Methods ...
}
```

```
List<Account> allAccounts = ... // init()
```

```
for (Account account: allAccounts) { // CPU dCache inefficient layout if
    if (!account.isActive())           // most of the accounts are active
        triggerEvent(...);
}
```

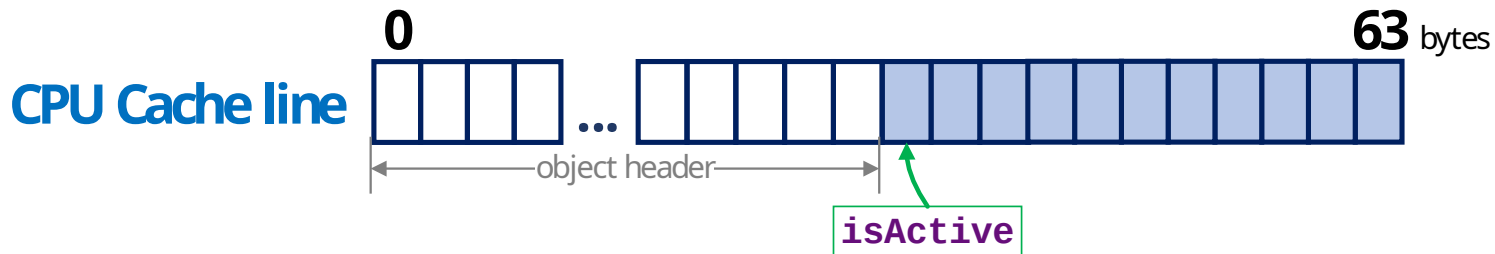


Data-Oriented Design

```
public class AccData {  
    private boolean[] areActive;  
    // ... related (e.g. used together) Fields ...  
}
```

```
AccData allAccData = ... // init()
```

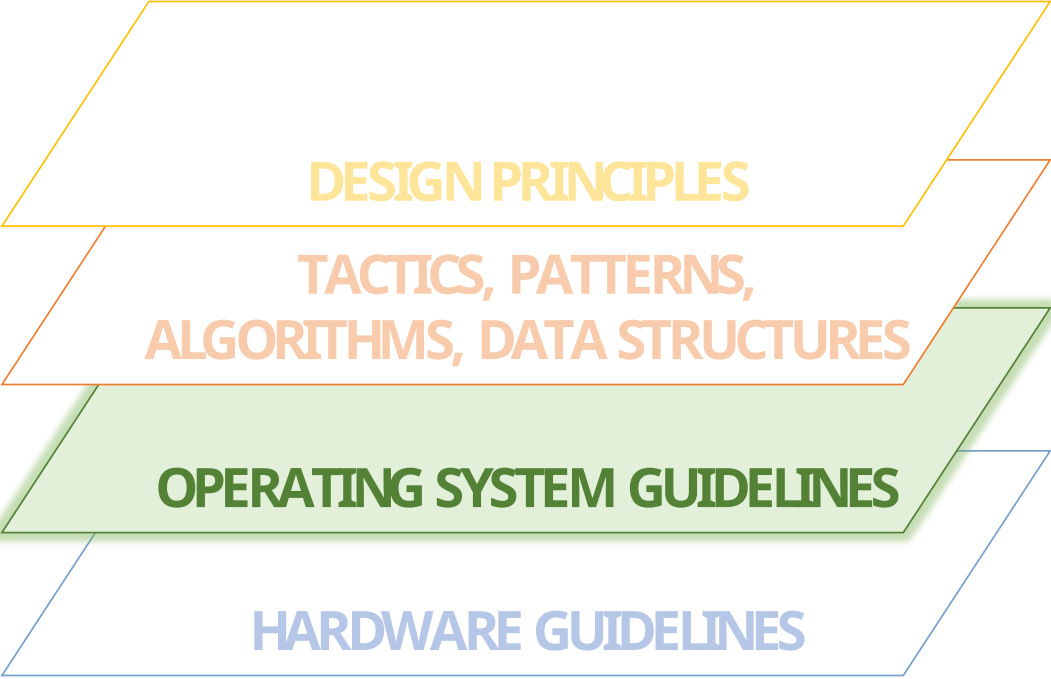
```
for (int i = 0; i < accData.areActive().length; i++) {  
    if (!accData.areActive[i])  
        triggerEvent(...);  
}
```



Data-Oriented Design

focuses on how data is read and written

- cache friendly access patterns -



DESIGN PRINCIPLES

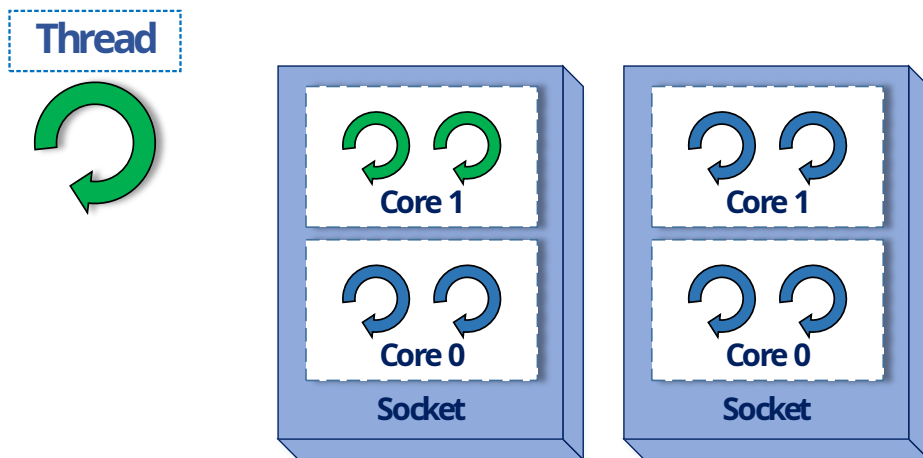
**TACTICS, PATTERNS,
ALGORITHMS, DATA STRUCTURES**

OPERATING SYSTEM GUIDELINES

HARDWARE GUIDELINES

Thread Affinity

Thread Affinity binds a thread to a CPU or a range of CPUs so that the thread will execute only on the designated CPU or CPUs rather than any CPU



Thread affinity takes advantages on CPU cache memory.
When a thread migrates from one processor to another all cache lines have to be moved.

Thread Affinity

In Java

Chronicle 

Thread Affinity

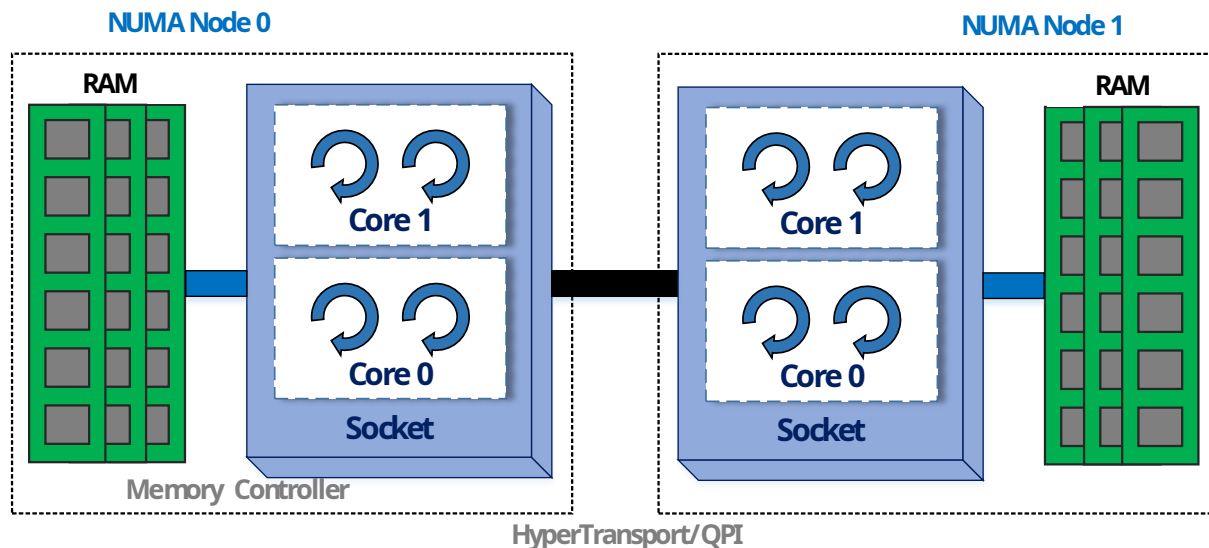
Minimise jitter for critical threads

In Linux

taskset - retrieve or set processes's CPU affinity

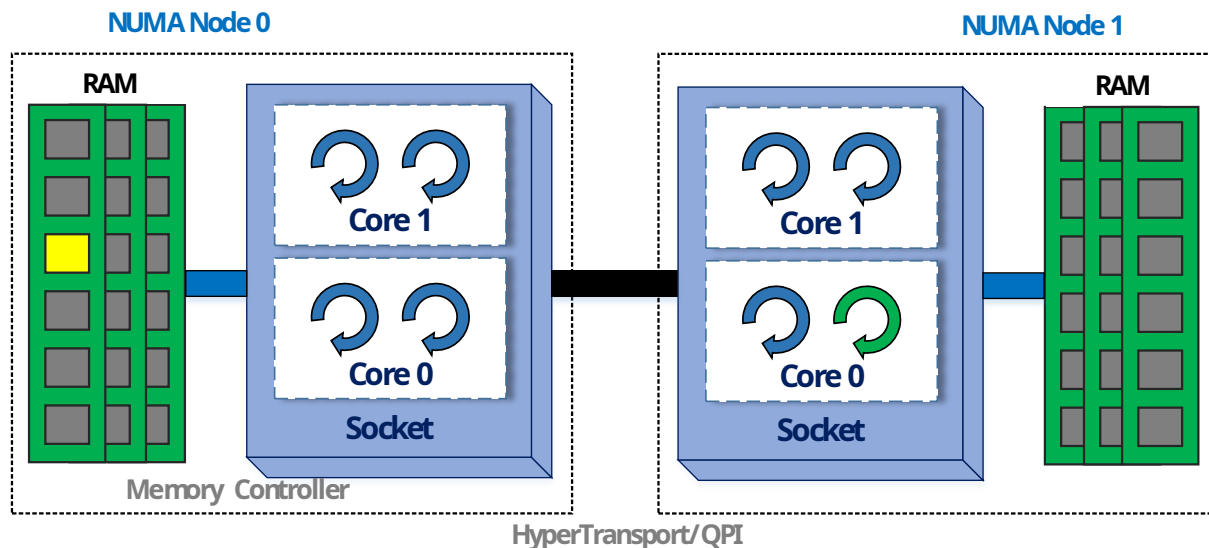
NUMA

Non-Uniform-Memory-Access (NUMA) is a memory design where the memory access time depends on the memory location relative to the processor



NUMA

Non-Uniform-Memory-Access (NUMA) is a memory design where the memory access time depends on the memory location relative to the processor



JVM NUMA-aware allocator has been implemented to take advantage of local memory

NUMA

In Java

-XX:+UseNUMA activates NUMA-aware collector

-XX:+UseParallelGC needs to be enabled as well (e.g. for Parallel Scavenger)

NUMA Performance Metrics

When evaluated against the SPEC JBB 2005 benchmark on an 8-chip Opteron machine, NUMA-aware systems showed the following performance increases:

- 32 bit – About 30 percent increase in performance with NUMA-aware allocator
- 64 bit – About 40 percent increase in performance with NUMA-aware allocator

In Linux

numactl - control NUMA policy for processes or shared memory

JEP 157: G1 GC: NUMA-Aware Allocation

<i>Author</i>	Y. Srinivas Ramakrishna
<i>Owner</i>	Jesper Wilhelmsson
<i>Created</i>	2011/07/28 20:00
<i>Updated</i>	2016/03/08 19:25
<i>Type</i>	Feature
<i>Status</i>	Candidate
<i>Component</i>	hotspot / gc
<i>Effort</i>	S
<i>Duration</i>	M
<i>Priority</i>	4

Summary

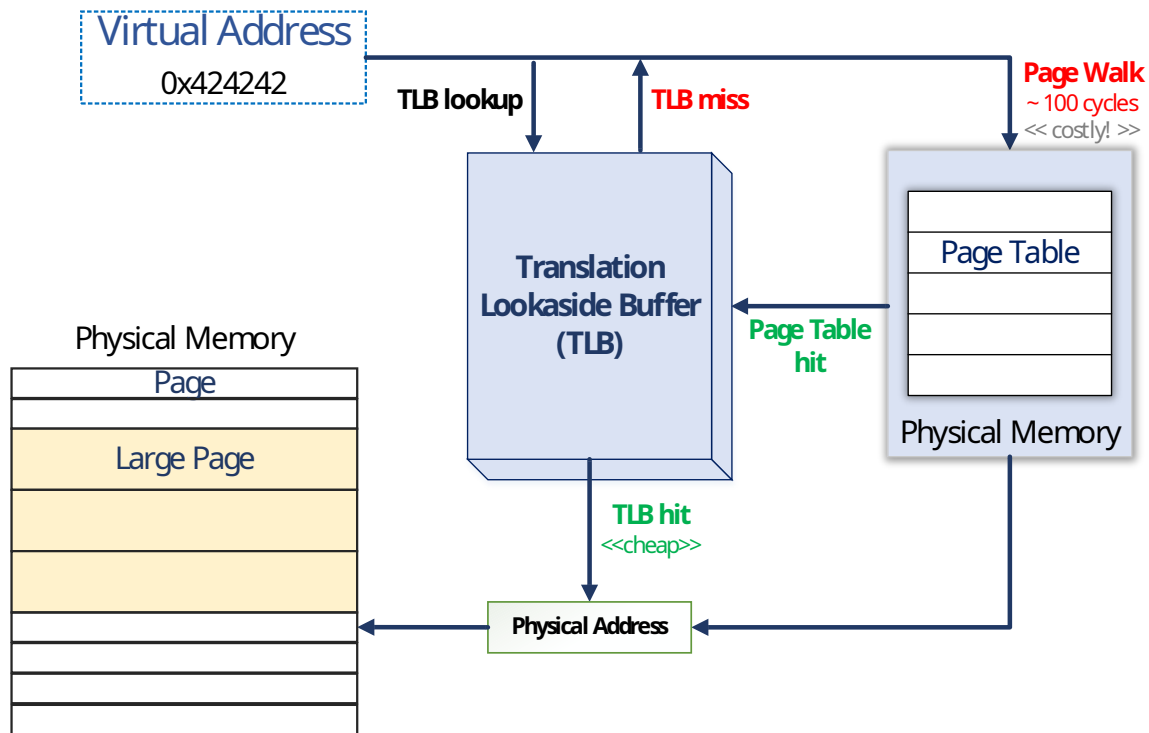
Enhance G1 to improve allocation performance on NUMA memory systems.

Non-Goals

Extend NUMA-awareness to work on any OS other than Linux and Solaris, which provide appropriate NUMA interfaces.

Large Pages

Using **Large Pages** the TLB can represent larger memory range hence reduces TLB misses and the number of page walks



Large Pages

In Java

-XX:+UseLargePages but it needs OS support

Operating system configuration changes to enable large pages:

- **Solaris:**
As of Solaris 9, which includes
- **Multiple Page Size Support (MPSS)**, no additional configuration is necessary.

Linux:

Large page support is included in 2.6 kernel. Some vendors have backported the code to their 2.4 based releases. To check if your system can support large page memory, try the following:

```
# cat /proc/meminfo | grep Huge
HugePages_Total: 0
HugePages_Free: 0
Hugepagesize: 2048 kB
#
```

- **Windows:**
Only Windows Server 2003 supports large page memory. In order to use it, the administrator must first assign additional privilege to the user who will be running the application:
 - select Control Panel -> Administrative Tools -> Local Security Policy
 - select Local Policies -> User Rights Assignment

Large Pages

Guidelines

- suitable for **intensive memory** applications
- with **large contiguous memory accesses**

Enable Large Pages when TLB misses and time spent in Page walk takes a significant amount of time



Large Pages

Not Recommended for ...

- short lived applications with **small working set**
- applications with **large** but **sparsely used heap**

RamFS & TmpFS

RamFS & TmpFS allocate a part of the physical memory to be used as a partition (e.g. write/read files).



Useful for applications which performs a lot disk reads/writes (e.g. logging, auditing)

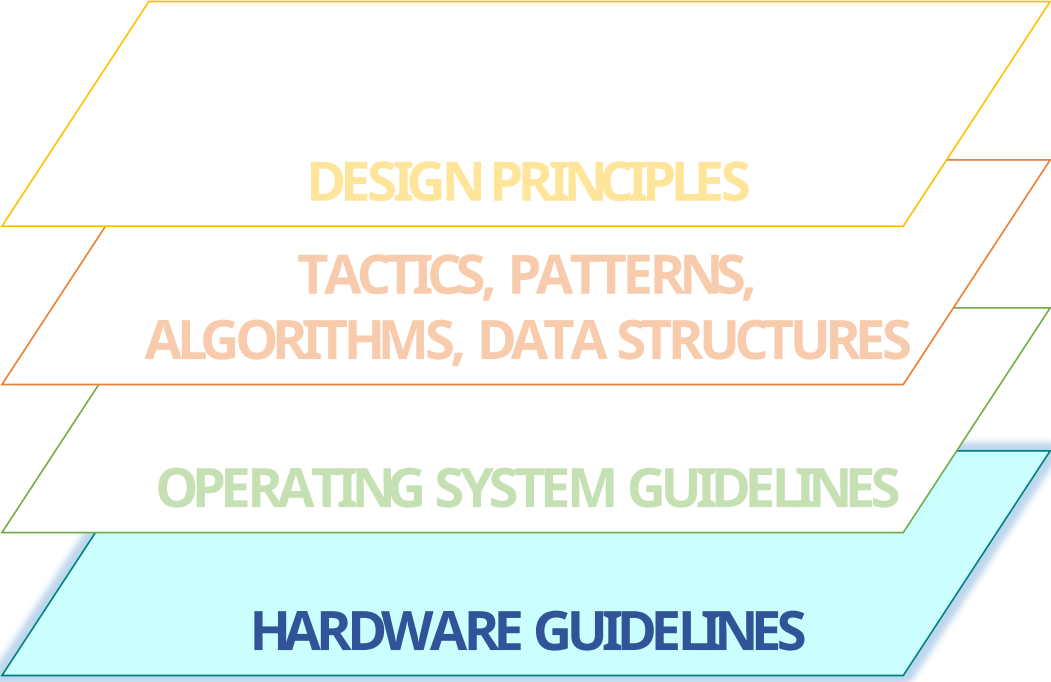
RamFS & TmpFS

	HDD (5,400RPM)		SSD		RAMFS	
Chunk	Read MB/s	Write MB/s	Read MB/s	Write MB/s	Read MB/s	Write MB/s
4K	128	99	964	742	7971	4420
512K	147	113	1021	788	10760	6045

higher is better

NB: higher read rates are caused by buffers/caches effect

Test scenario: sequentially reading/writing 8GB in chunks of 4KB/512 KB on HDD/SSD/RAMFS



DESIGN PRINCIPLES

**TACTICS, PATTERNS,
ALGORITHMS, DATA STRUCTURES**

OPERATING SYSTEM GUIDELINES

HARDWARE GUIDELINES

False Sharing

False Sharing is purely a CPU Cache issue

```
public class FalseSharing {  
    public int X;  
    public int Y;  
}
```

```
FalseSharing sharedInstance = new FalseSharing();
```

Thread 1

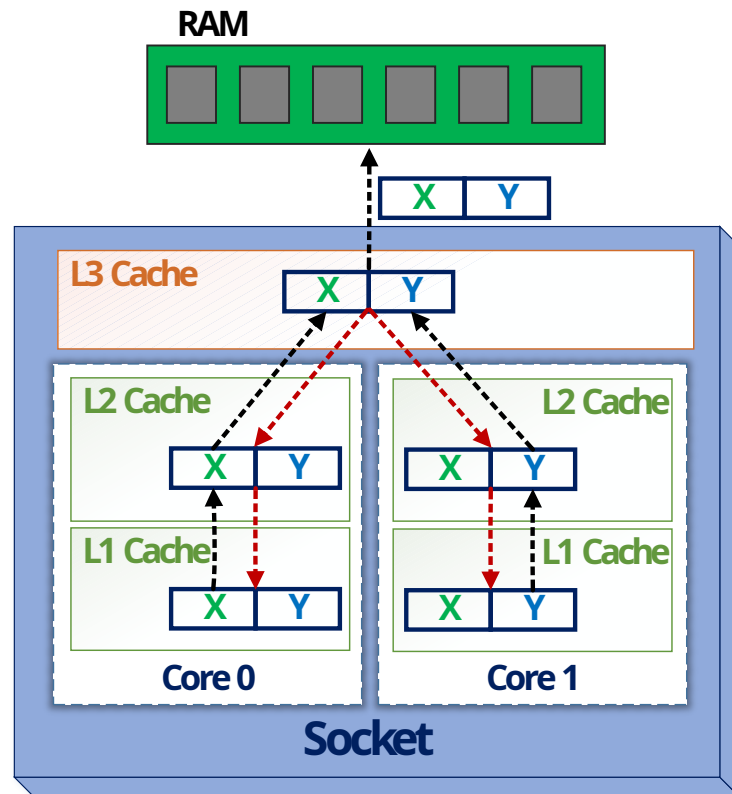


```
void incrementX () {  
    sharedInstance.X ++  
}
```

Thread 2



```
void incrementY () {  
    sharedInstance.Y ++  
}
```



-----> Request for Ownership (I -> M)
-----> Update

False Sharing

Guidelines

- independent values sits on the **same cache line**
- different cores **concurrently access** that line
- there is at **least one writer** thread
- **high frequency** of writing/reading

False Sharing

In Java

@Contended annotation pads fields so they will sit on different cache lines

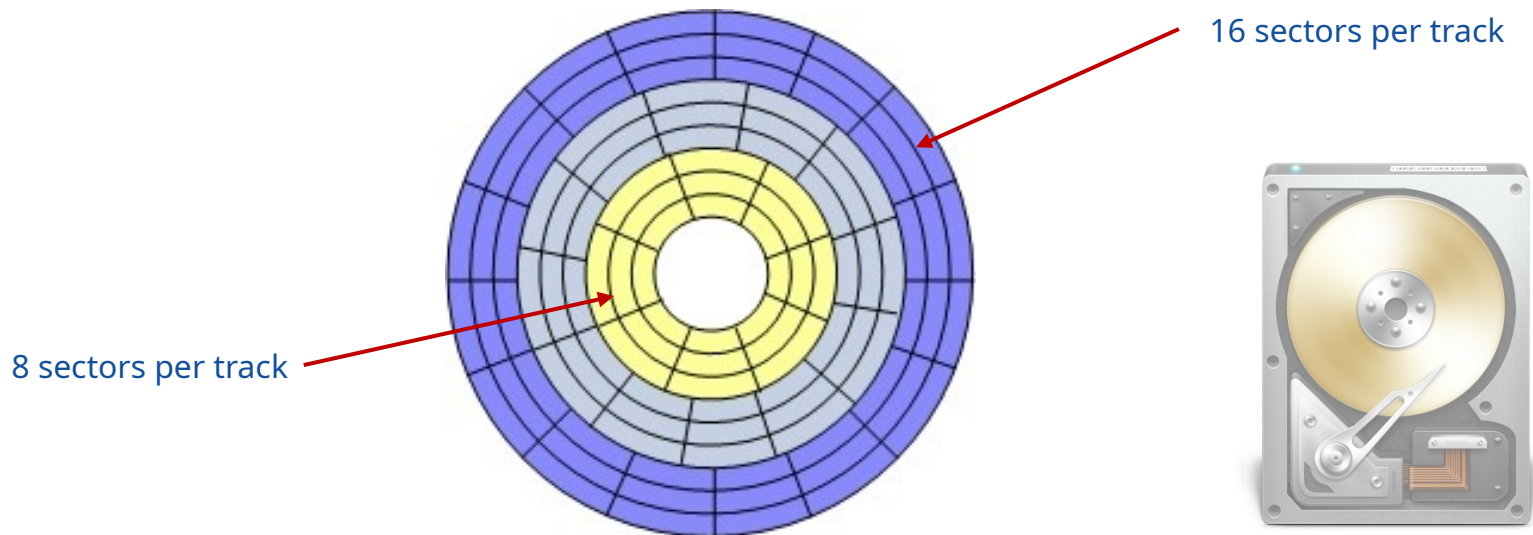
```
public class FalseSharing {  
    public int X;  
  
    @sun.misc.Contended  
    public int Y;  
}
```

Threads	Baseline (# of ops / ms)	@Contended (# of ops / ms)
2	478.051	1,378.564

higher is better

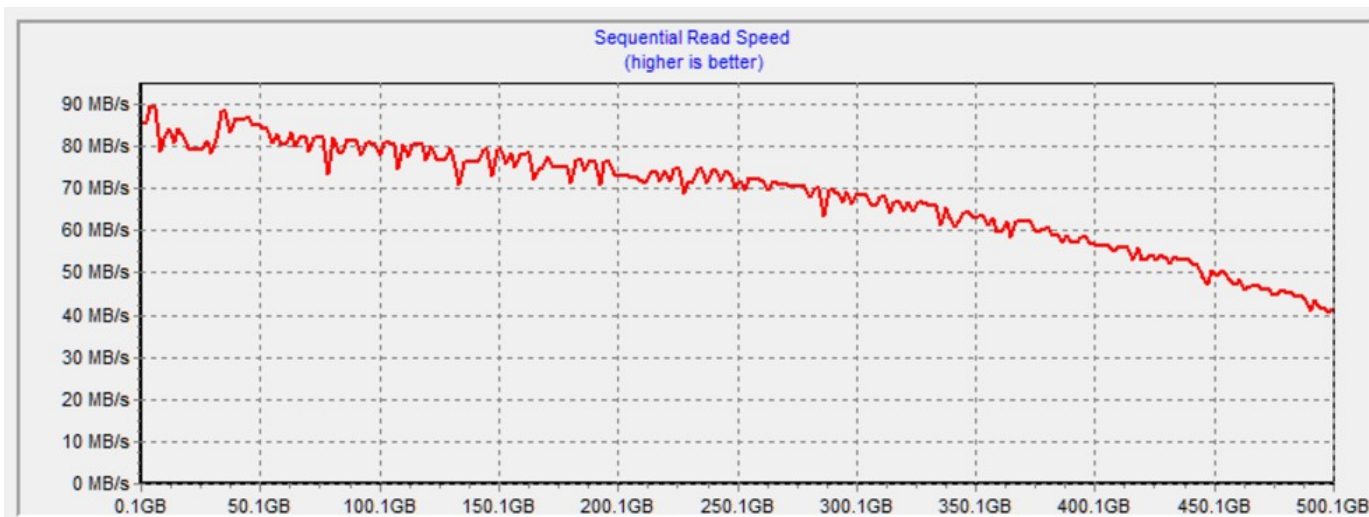
Zero Bit Recording

Zero Bit Recording (ZBR) stores more sectors per track on outer tracks than on inner tracks on HDD



There are more bits stored in outside tracks (e.g. more sectors per track) hence the outer tracks have highest transfer rate.

Zero Bit Recording



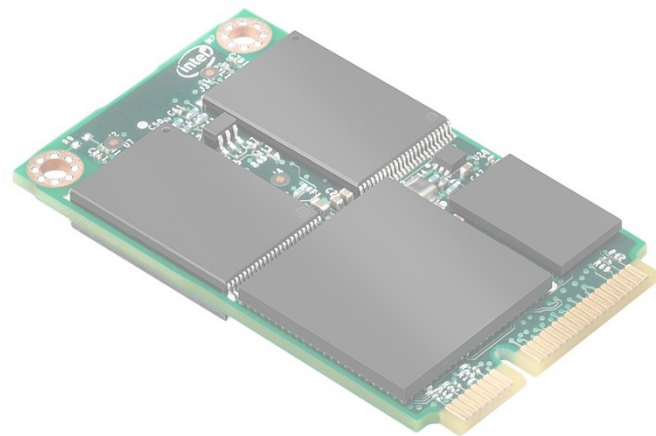
[Source: <http://www.overclockers.com/forums/showthread.php/678783-What-are-the-standart-write-read-speeds-for-sata-HDD>]

Solid State Drive

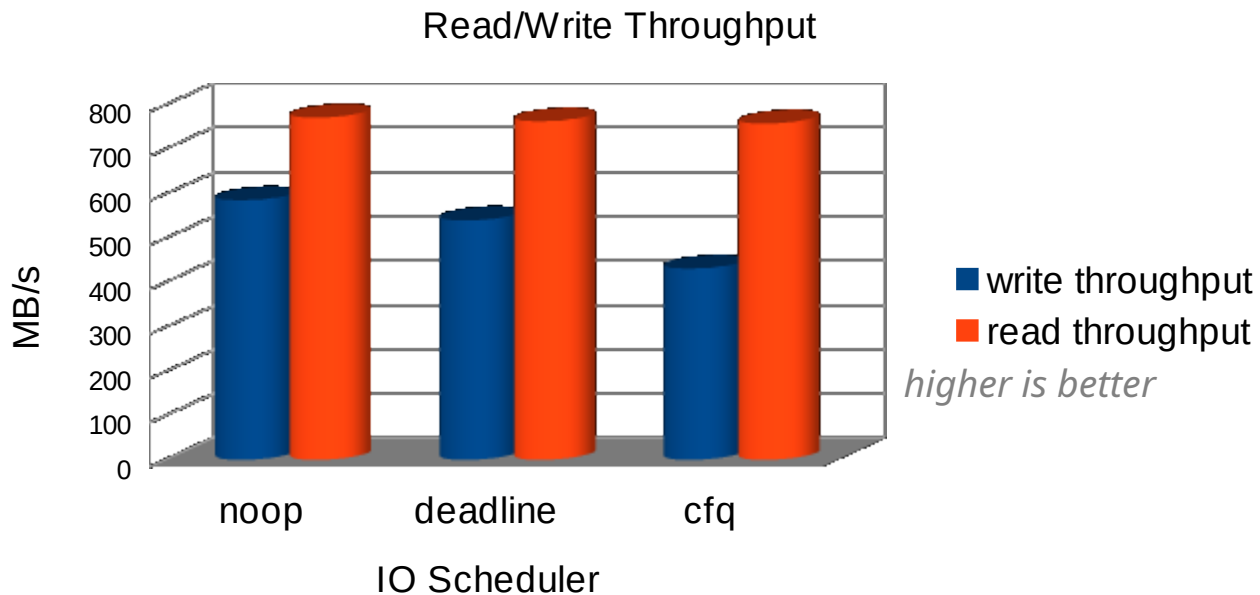
Enable TRIM

I/O Scheduler

Enable write caching

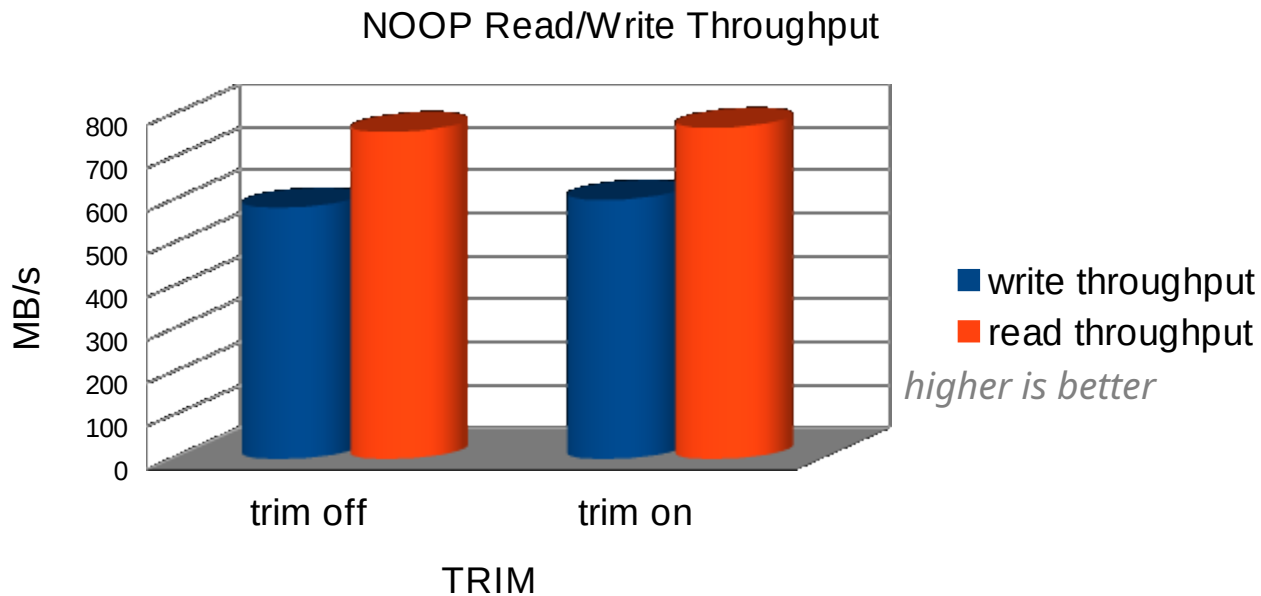


Solid State Drive



Test scenario: sequentially writing/reading 32GB in chunks of 512 KB on SSD

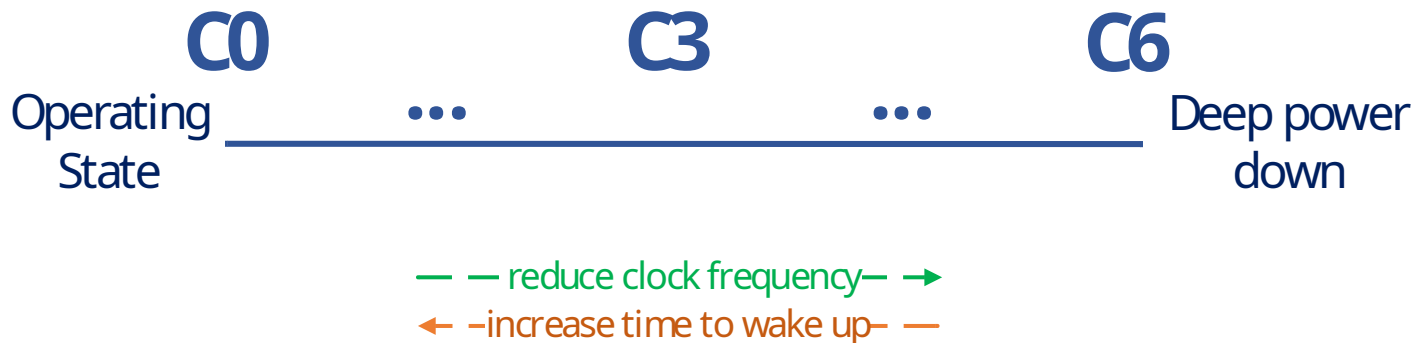
Solid State Drive



Test scenario: sequentially writing/reading 32GB in chunks of 512 KB on SSD

C-States

C-States cut the clock signal (CPU power) to save energy but ... **more time is required for CPU to "wake up"** and again be 100% operational



Consider disabling C-States to maintain a steady high CPU performance

My Latency Hierarchical Model

Ultra-low Latency

Thread affinity, NUMA, large pages, false sharing, Data-Oriented Design, C-states, CPU caches

Low Latency

Memory access patterns, lock free, asynchronous processing, stateless, RamFS/TmpFS, GC and object lifecycle tuning

Affordable Latency

Data structures, algorithms complexities, batching, caching

Performance is not an ASR

Clean code, cyclomatic complexity, cohesion, abstractions

Performance is simple, you just have to be aware of everything!

Ionuț Baloșin





Prague, 19-20 October 2017

www.luxoft.com

Thank You

Ionuț Baloșin

Software Architect

 **@ionutbalosin**

Bibliography

- ✓ Cpu Caches and Why You Care - Scott Meyers
- ✓ CPU caches - Ulrich Drepper
- ✓ Async or Bust!? - Todd Montgomery
- ✓ <http://mechanical-sympathy.blogspot>
- ✓ <http://vanillajava.blogspot>
- ✓ <http://preshing.com/20120612/an-introduction-to-lock-free-programming>
- ✓ <http://heather.cs.ucdavis.edu/~matloff/50/PLN/lock.pdf>
- ✓ <http://www.glennklockwood.com/hpc-howtos/process-affinity.html>
- ✓ <http://docs.oracle.com/javase/7/docs/technotes/guides/vm/performance-enhancements-7.html>
- ✓ <http://cenalulu.github.io/linux/huge-page-on-numa>
- ✓ <http://www.thegeekstuff.com/2008/11/overview-of-ramfs-and-tmpfs-on-linux>



Prague, 19-20 October 2017

www.luxoft.com

Thank You

Ionuț Baloșin

Software Architect

 @ionutbalosin



Latency Numbers Every Programmer Should Know

Latency Comparison Numbers

by **Jeff Dean** / originally by **Peter Norvig**

L1 cache reference	0.5	ns			
Branch mispredict	5	ns			
L2 cache reference	7	ns			14x L1 cache
Mutex lock/unlock	25	ns			
Main memory reference	100	ns			20x L2 cache, 200x L1 cache
Compress 1K bytes with Zippy	3,000	ns	3	us	
Send 1K bytes over 1 Gbps network	10,000	ns	10	us	
Read 4K randomly from SSD*	150,000	ns	150	us	~1GB/sec SSD
Read 1 MB sequentially from memory	250,000	ns	250	us	
Round trip within same datacenter	500,000	ns	500	us	
Read 1 MB sequentially from SSD*	1,000,000	ns	1,000	us	1 ms ~1GB/sec SSD, 4X memory
Disk seek	10,000,000	ns	10,000	us	10 ms 20x datacenter roundtrip
Read 1 MB sequentially from disk	20,000,000	ns	20,000	us	20 ms 80x memory, 20X SSD
Send packet CA->Netherlands->CA	150,000,000	ns	150,000	us	150 ms

Notes

1 ns = 10^{-9} seconds

1 us = 10^{-6} seconds = 1,000 ns

1 ms = 10^{-3} seconds = 1,000 us = 1,000,000 ns